```
/* ================= EV Charging Station Route Planner================= */

/* ---- Dynamic facts-----*/
:- dynamic(blocked_road/2).
:- dynamic(station_down/1).

/* ---- Vehicle settings-----*/
battery_capacity(220).
speed_kmph(70).

/* ---- Town locations (Name, Lat, Lon)-----*/
location(colombo,      6.93, 79.86).
location(kadawatha,    6.99, 79.95).
location(kegalle,      7.25, 80.35).
location(mawanella,    7.25, 80.44).
location(kandy,        7.29, 80.64).
location(kurunegala,   7.49, 80.36).
location(dambulla,     7.87, 80.65).
location(anuradhapura, 8.31, 80.40).
location(galle,        6.05, 80.22).
location(matara,       5.95, 80.54).
location(ratnapura,    6.68, 80.40).
location(nuwaraeliya,  6.97, 80.77).
```

```
/* ---- Road network-----*/
road(colombo,    kadawatha,    12).
road(kadawatha,  kegalle,      65).
road(kegalle,    mawanella,    20).
road(mawanella,  kandy,        26).
road(kandy,      kurunegala,   42).
road(kurunegala, dambulla,     45).
road(dambulla,   anuradhapura, 65).
road(colombo,    ratnapura,    101).
road(ratnapura,  nuwaraeliya,  65).
road(kandy,      nuwaraeliya,  77).
road(colombo,    galle,        119).
road(galle,      matara,       45).

/* ---- Charging stations-----*/
charging_station(kegalle,    3).
charging_station(kandy,      4).
charging_station(kurunegala, 2).
charging_station(dambulla,   3).
charging_station(ratnapura,  2).
charging_station(galle,      4).
```

```
/* ---- Road / charger blocking controls ---- */
block_road(A, B) :-
   (  is_blocked_road(A, B)
   ->  format("Road ~w - ~w is already blocked.~n", [A, B])
   ;  assertz(blocked_road(A, B)),
      format("Road ~w - ~w is now BLOCKED.~n", [A, B])
   ).

unblock_road(A, B) :-
   retractall(blocked_road(A, B)),
   retractall(blocked_road(B, A)),
   format("Road ~w - ~w is now OPEN.~n", [A, B]).

block_station(Town) :-
   (  station_down(Town)
   ->  format("Charger at ~w is already down.~n", [Town])
   ;  assertz(station_down(Town)),
      format("Charger at ~w is now OUT OF SERVICE.~n", [Town])
   ).

unblock_station(Town) :-
   retractall(station_down(Town)),
   format("Charger at ~w is now WORKING.~n", [Town]).

is_blocked_road(A, B) :- blocked_road(A, B).
is_blocked_road(A, B) :- blocked_road(B, A).
```

```
list_constraints :-
    findall(A-B, blocked_road(A, B), Roads),
    findall(T, station_down(T), Stations),
    format("Blocked roads: ~w~n", [Roads]),
    format("Chargers down: ~w~n", [Stations]).

connected(X, Y, D) :- road(X, Y, D), \+ is_blocked_road(X, Y).
connected(X, Y, D) :- road(Y, X, D), \+ is_blocked_road(X, Y).

/* ---- Successor moves (drive / charge)----- */
successors(state(Town, Range), Results) :-
    findall(
        succ(NewState, Desc, TimeMin),
        (
            (  connected(Town, Next, Dist),
               Dist =< Range,
               Range2 is Range - Dist,
               NewState = state(Next, Range2),
               speed_kmph(Speed),
               TimeMin is Dist / Speed * 60,
               format(atom(Desc), "Drive ~w -> ~w (~wkm, ~1f min)",
                    [Town, Next, Dist, TimeMin])
            )
        ;
            (  charging_station(Town, RatePerMin),
```

```
            \+ station_down(Town),
            battery_capacity(Cap),
            Range < Cap,
            NeedKm is Cap - Range,
            TimeMin is NeedKm / RatePerMin,
            NewState = state(Town, Cap),
            format(atom(Desc), "Charge at ~w (+~wkm, ~1f min)",
                [Town, NeedKm, TimeMin])
        )
      ),
      Results
  ).

/* ---- A* heuristic (straight-line distance/time)----- */
haversine(A, B, Km) :-
  location(A, Lat1, Lon1), location(B, Lat2, Lon2),
  R = 6371,
  DLat is (Lat2 - Lat1) * pi / 180,
  DLon is (Lon2 - Lon1) * pi / 180,
  RLat1 is Lat1 * pi / 180, RLat2 is Lat2 * pi / 180,
  X is sin(DLat / 2) ** 2 + cos(RLat1) * cos(RLat2) * sin(DLon / 2) ** 2,
  C is 2 * atan2(sqrt(X), sqrt(1 - X)),
  Km is R * C.

heuristic_time(Town, Goal, H) :-
  haversine(Town, Goal, Dist),
```

```
    speed_kmph(Speed),
    H is Dist / Speed * 60.

/* ---- BFS: fewest actions ---- */
bfs(Start, Goal, Steps, TotalTime, Expanded) :-
    battery_capacity(Cap),
    bfs_loop([node(state(Start, Cap), [])], Goal,
             [state(Start, Cap)], 0, RevSteps, Expanded),
    reverse(RevSteps, Steps),
    sum_time(Steps, TotalTime).

bfs_loop([node(state(Goal, _), Steps) | _], Goal, _Visited, Exp0, Steps, Exp) :-
    !, Exp is Exp0 + 1.
bfs_loop([node(State, Steps) | OtherPaths], Goal, Visited, Exp0, Result, Exp) :-
    Exp1 is Exp0 + 1,
    successors(State, Succs),
    findall(node(NewState, [Desc-TimeMin | Steps]),
            ( member(succ(NewState, Desc, TimeMin), Succs),
              \+ member(NewState, Visited) ),
            NewPaths),
    findall(S, member(node(S, _), NewPaths), NewStates),
    append(Visited, NewStates, Visited1),
    append(OtherPaths, NewPaths, Queue1),
    bfs_loop(Queue1, Goal, Visited1, Exp1, Result, Exp).
```

```
/* ---- DFS: first feasible route ---- */
dfs(Start, Goal, Steps, TotalTime, Expanded) :-
    battery_capacity(Cap),
    dfs_search(state(Start, Cap), Goal, [state(Start, Cap)], [], RevSteps, 0, Expanded),
    reverse(RevSteps, Steps),
    sum_time(Steps, TotalTime).

dfs_search(state(Goal, _), Goal, _Visited, Steps, Steps, Exp0, Exp) :- !, Exp is Exp0 + 1.
dfs_search(State, Goal, Visited, StepsSoFar, Steps, Exp0, Exp) :-
    Exp1 is Exp0 + 1,
    successors(State, Succs),
    member(succ(NewState, Desc, TimeMin), Succs),
    \+ member(NewState, Visited),
    dfs_search(NewState, Goal, [NewState | Visited],
            [Desc-TimeMin | StepsSoFar], Steps, Exp1, Exp).

/* ---- A*: minimum total trip time----- */
astar(Start, Goal, Steps, TotalTime, Expanded) :-
    battery_capacity(Cap),
    heuristic_time(Start, Goal, H0),
    astar_loop([node(H0, 0, state(Start, Cap), [])], Goal, [],
            RevSteps, TotalTime, 0, Expanded),
    reverse(RevSteps, Steps).

astar_loop(Open, Goal, _Closed, Steps, G, Exp0, Exp) :-
     select_min(Open, node(_, G, state(Goal, _), Steps), _),
```

```
    !, Exp is Exp0 + 1.
astar_loop(Open, Goal, Closed, Steps, TotalTime, Exp0, Exp) :-
    select_min(Open, node(_, G, State, StepsSoFar), Open1),
    Exp1 is Exp0 + 1,
    (   memberchk(State, Closed)
    ->  astar_loop(Open1, Goal, Closed, Steps, TotalTime, Exp1, Exp)
    ;   successors(State, Succs),
        findall(node(F1, G1, NewState, [Desc-TimeMin | StepsSoFar]),
                ( member(succ(NewState, Desc, TimeMin), Succs),
                  \+ memberchk(NewState, Closed),
                  G1 is G + TimeMin,
                  NewState = state(NewTown, _),
                  heuristic_time(NewTown, Goal, H1),
                  F1 is G1 + H1
                ),
                NewNodes),
        append(Open1, NewNodes, Open2),
        astar_loop(Open2, Goal, [State | Closed], Steps, TotalTime, Exp1, Exp)
    ).


select_min([N], N, []) :- !.
select_min([N | Ns], Min, Rest) :-
    select_min(Ns, Min0, Rest0),
    arg(1, N, F), arg(1, Min0, F0),
    (   F =< F0
    ->  Min = N, Rest = Ns
```

```
    ;   Min = Min0, Rest = [N | Rest0]
    ).

/* ---- Helpers ---- */
sum_time(Steps, Total) :-
    findall(T, member(_-T, Steps), Ts),
    sum_list(Ts, Total).

count_charges(Steps, N) :-
    findall(1, ( member(Desc-_, Steps), sub_atom(Desc, 0, 6, _, 'Charge') ), L),
    length(L, N).

print_steps([]) :- !.
print_steps(Steps) :- forall(member(Desc-_, Steps), format("     ~w~n", [Desc])).

/* ---- Run & compare algorithms ----  */
run_bfs(Start, Goal) :-
    get_time(T0),
    ( bfs(Start, Goal, Steps, TripTime, Expanded) -> true
    ; Steps = [], TripTime = 0, Expanded = 0 ),
    get_time(T1), Ms is (T1 - T0) * 1000,
    length(Steps, N), count_charges(Steps, C),
    format("~nBFS (fewest actions):~n"), print_steps(Steps),
    format("  TOTAL: ~1f min | ~w actions | ~w charge stops | ~w nodes expanded | ~3f ms~n",
          [TripTime, N, C, Expanded, Ms]).
run_dfs(Start, Goal) :-
```

```
    get_time(T0),
    ( dfs(Start, Goal, Steps, TripTime, Expanded) -> true
    ; Steps = [], TripTime = 0, Expanded = 0 ),
    get_time(T1), Ms is (T1 - T0) * 1000,
    length(Steps, N), count_charges(Steps, C),
    format("~nDFS (first feasible route):~n"), print_steps(Steps),
    format("  TOTAL: ~1f min | ~w actions | ~w charge stops | ~w nodes expanded | ~3f ms~n",
        [TripTime, N, C, Expanded, Ms]).

run_astar(Start, Goal) :-
    get_time(T0),
    ( astar(Start, Goal, Steps, TripTime, Expanded) -> true
    ; Steps = [], TripTime = 0, Expanded = 0 ),
    get_time(T1), Ms is (T1 - T0) * 1000,
    length(Steps, N), count_charges(Steps, C),
    format("~nA* (minimum total trip time):~n"), print_steps(Steps),
    format("  TOTAL: ~1f min | ~w actions | ~w charge stops | ~w nodes expanded | ~3f ms~n",
        [TripTime, N, C, Expanded, Ms]).

compare_algorithms(Start, Goal) :-
    battery_capacity(Cap),
    format("~n=== EV Trip: ~w -> ~w (battery ~w km full charge) ===~n",
        [Start, Goal, Cap]),
    run_bfs(Start, Goal),
    run_dfs(Start, Goal),
    run_astar(Start, Goal).
```